

Derin Öğrenme ile Beyin Tümörü Sınıflandırma

https://github.com/TubaGk/brain_tumor_detector

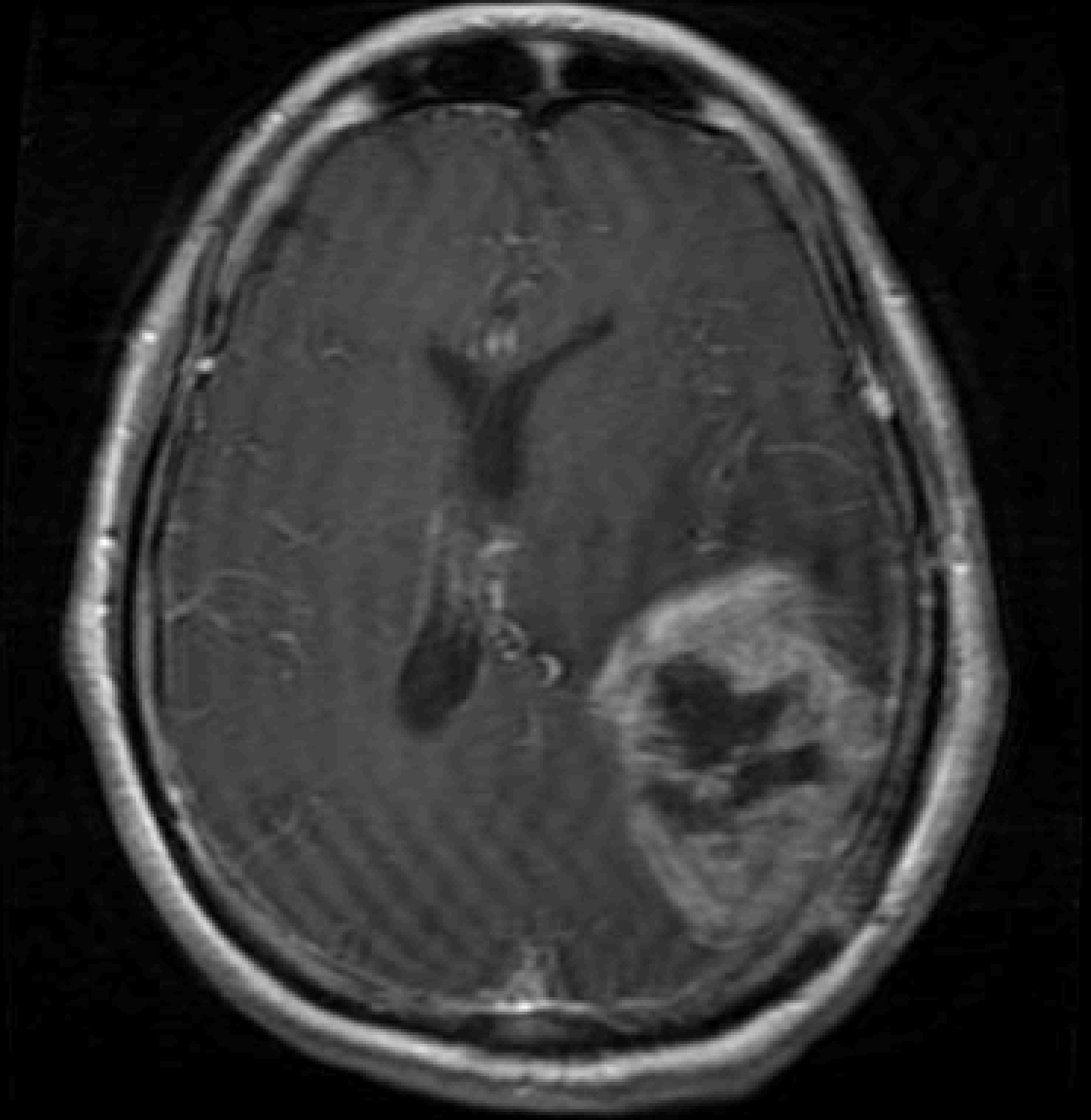
1. Glioma (Glial T m rler)

MR G r nt s ndeki  zellikleri:

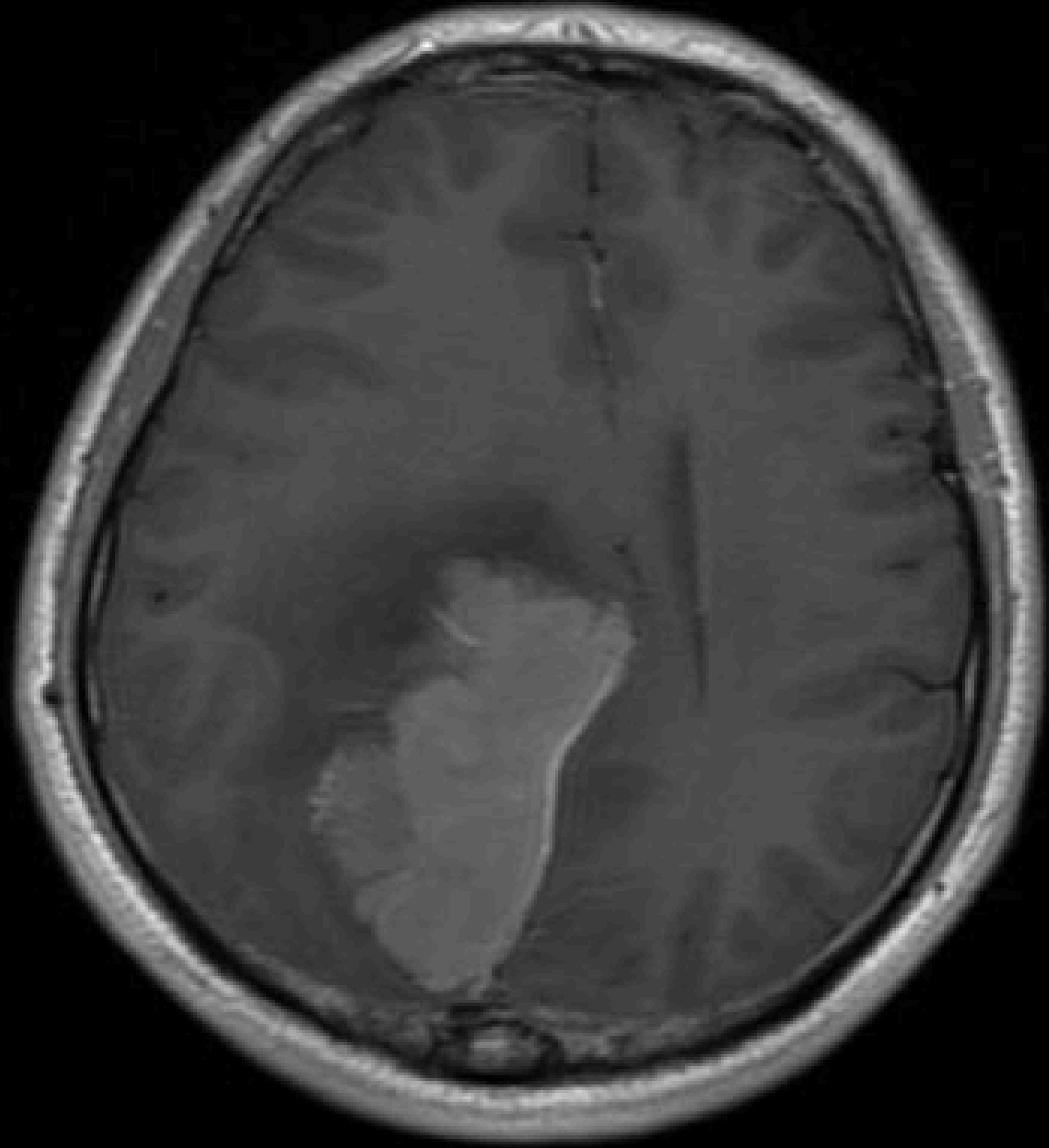
- Genellikle beyin dokusunun i inde yer alır.
- D zensiz  ekilli, belirsiz sınırlı olabilir.
- Heterojen yapı g sterir (farklı yo unluklar i erebilir).
-  dem ( işilik) ve nekroz ( l  doku) da olabilir.

Tespit Etme Y ntemi:

- Beyin dokusunun i  kısmındaki anormal sinyaller model tarafından   renilir.
-  zellikle T1 ve T2 sekanslarında kontrast farklılıkları modele fikir verir.



2. Meningioma (Beyin Zarı Tümörü)



MR Görüntüsündeki Özellikleri:

- Beyin yüzeyine yakın, dışı yakın konumlanır.
- Yuvarlak, düzenli sınırları olan kitle şeklinde görülür.
- Genellikle kontrast madde ile parlaklaşır (enhancing).
- Sıklıkla kemik yapıya baskı yapabilir.

Tespit Etme Yöntemi:

- Model, korteks kenarında net sınırlı bir kitle olup olmadığını algılar.
- Kontrast farklılıklarını ve lokalizasyonu dikkate alır.

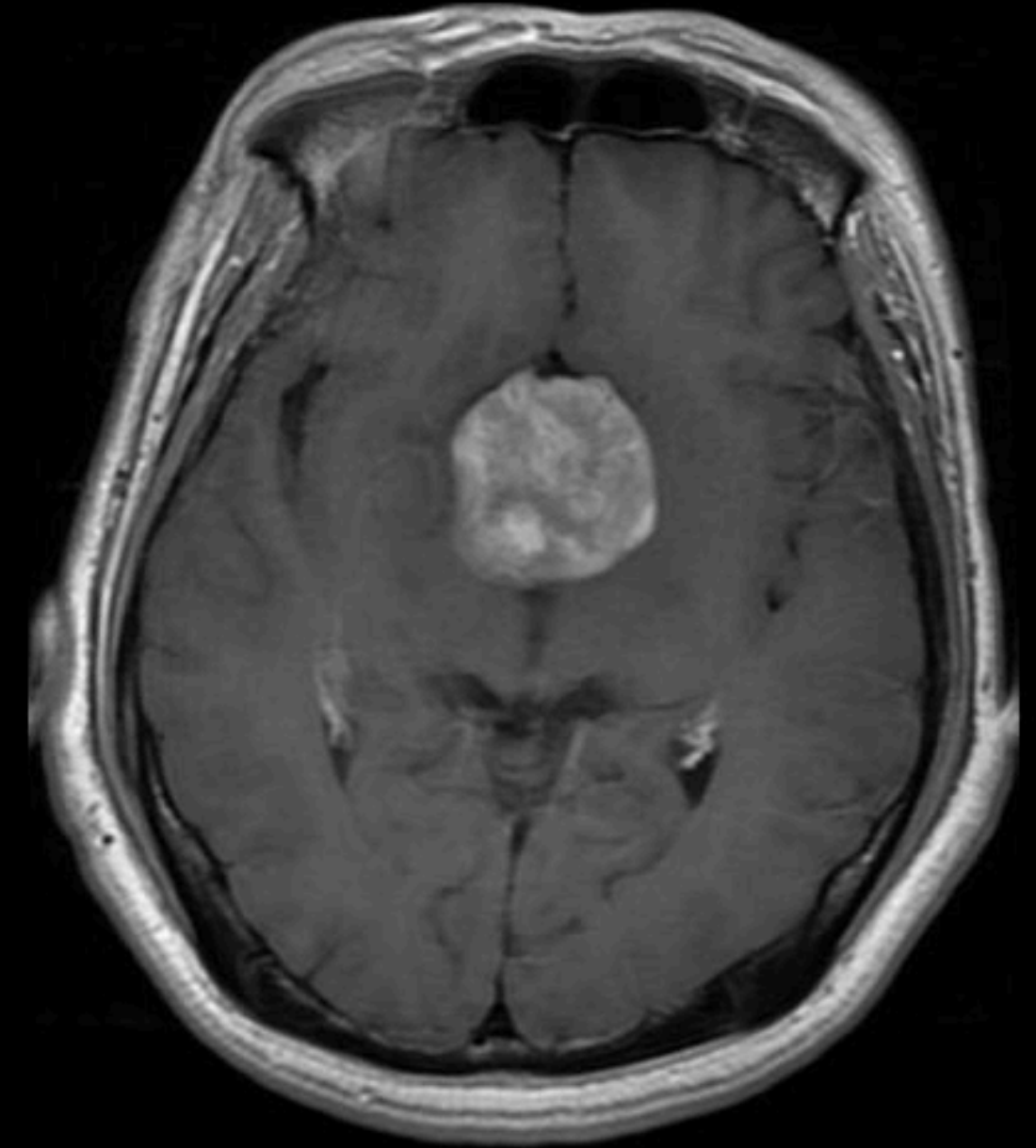
3. Pituitary Tumor (Hipofiz Tümörü)

MR Görüntüsündeki Özellikleri:

- Beynin tam ortasında, sella turcica denen kemik yapının içinde bulunur.
- Küçük, simetrik olabilir; ama büyüyünce çevresine baskı yapar.
- Görme sinirine yakın olduğu için büyüdüğünde bu siniri iter.

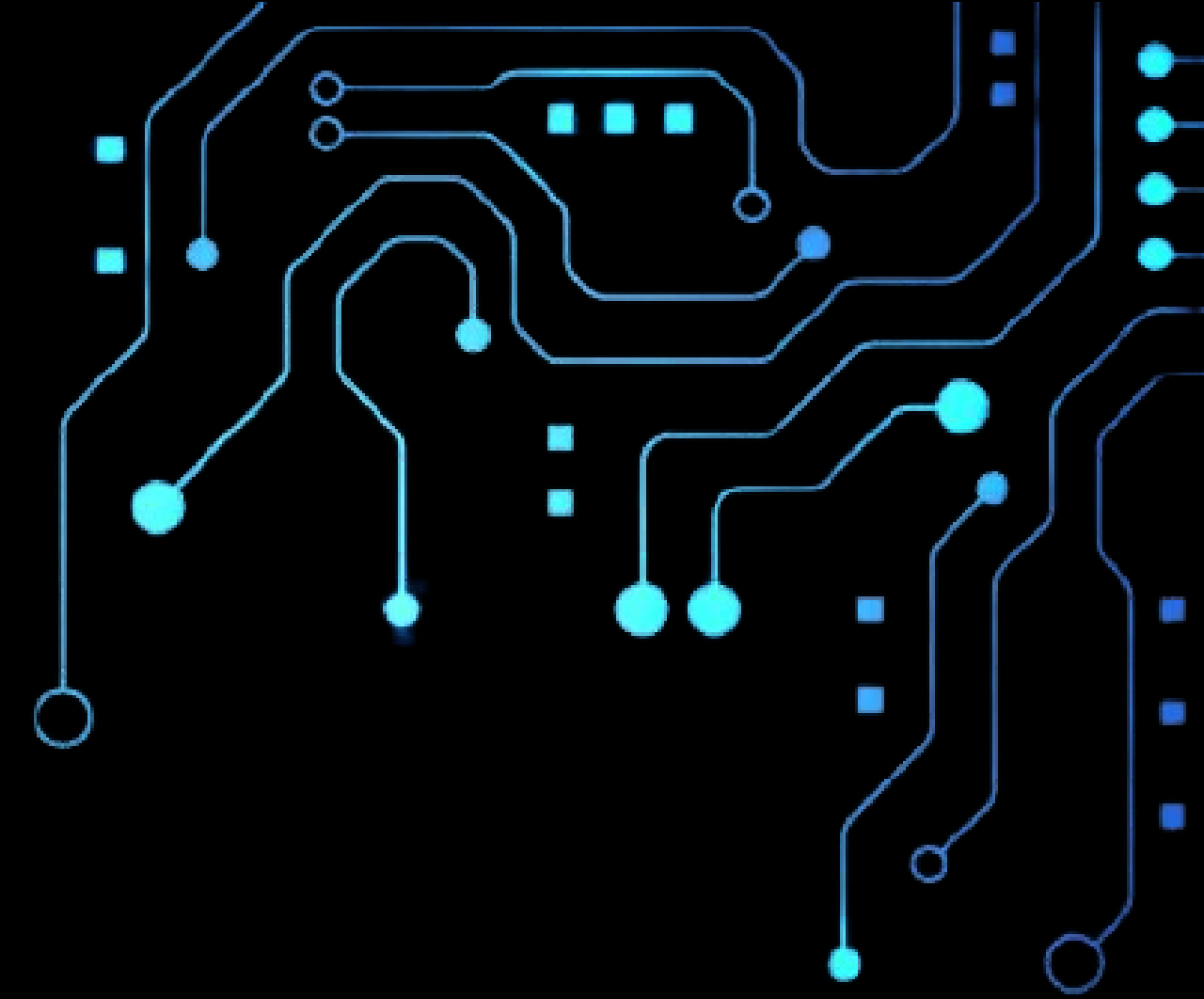
Tespit Etme Yöntemi:

- Model, özellikle orta bölgede (hipofiz alanı) odaklanır.
- Küçük boyutta bile olsa simetrik yapının bozulduğunu fark edebilir.



Model Mimarisi

model.py



Basit bir Convolutional Neural Network (CNN) mimarisi kullandık.

Katman yapısı:

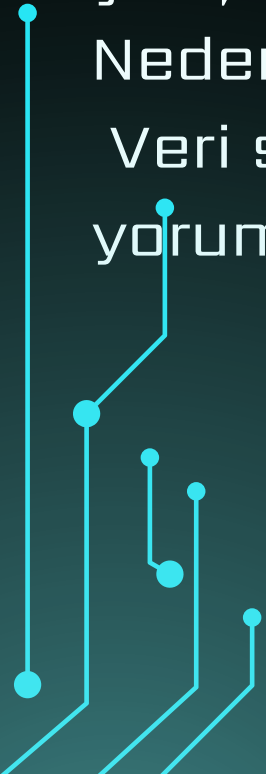
- 2 adet Conv2D + ReLU + MaxPool katmanı
- 2 adet Fully Connected (FC) katman

Giriş Boyutu: 224x224 RGB

Çıkış: 4 sınıf softmax skorları

Neden Basit CNN?

Veri seti boyutuna göre fazla derin olmayan, hızlı eğitilebilen ve yorumlaması kolay bir mimari tercih ettik.



```
class CNNModel(nn.Module):
```

```
    👤 zaylmzdr +1
```

```
def __init__(self, num_classes=4):
```

```
    super(CNNModel, self).__init__()
```

```
    self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
```

```
    self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
    self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
```

```
    # Fully connected layer giris boyutunu dummy input ile hesapla
```

```
    with torch.no_grad():
```

```
        dummy = torch.zeros(1, 3, 224, 224) # 224x224 giris varsayıldı
```

```
        x = self.pool(F.relu(self.conv1(dummy)))
```

```
        x = self.pool(F.relu(self.conv2(x)))
```

```
        n_size = x.numel() # Tüm eleman sayısı (batch hariç)
```

```
    self.fc1 = nn.Linear(n_size, out_features=128)
```

```
    self.fc2 = nn.Linear(in_features=128, num_classes)
```

Eğitim Süreci

train.py

Kullanılan Optimizasyon:

- Loss Function: CrossEntropyLoss
- Optimizer: Adam
- Adam, özellikle sınırlı veri ve düşük batch size ile iyi performans gösterir. Öğrenme oranı: 0.001

Epoch Sayısı: 10

Bu değer, hem overfitting'i önlemek hem de yeterli performansa ulaşmak için deneyerek belirlenmiştir.

Eğitim Aşamasında Kayıt Edilenler:

Train/Val Loss

Train/Val Accuracy

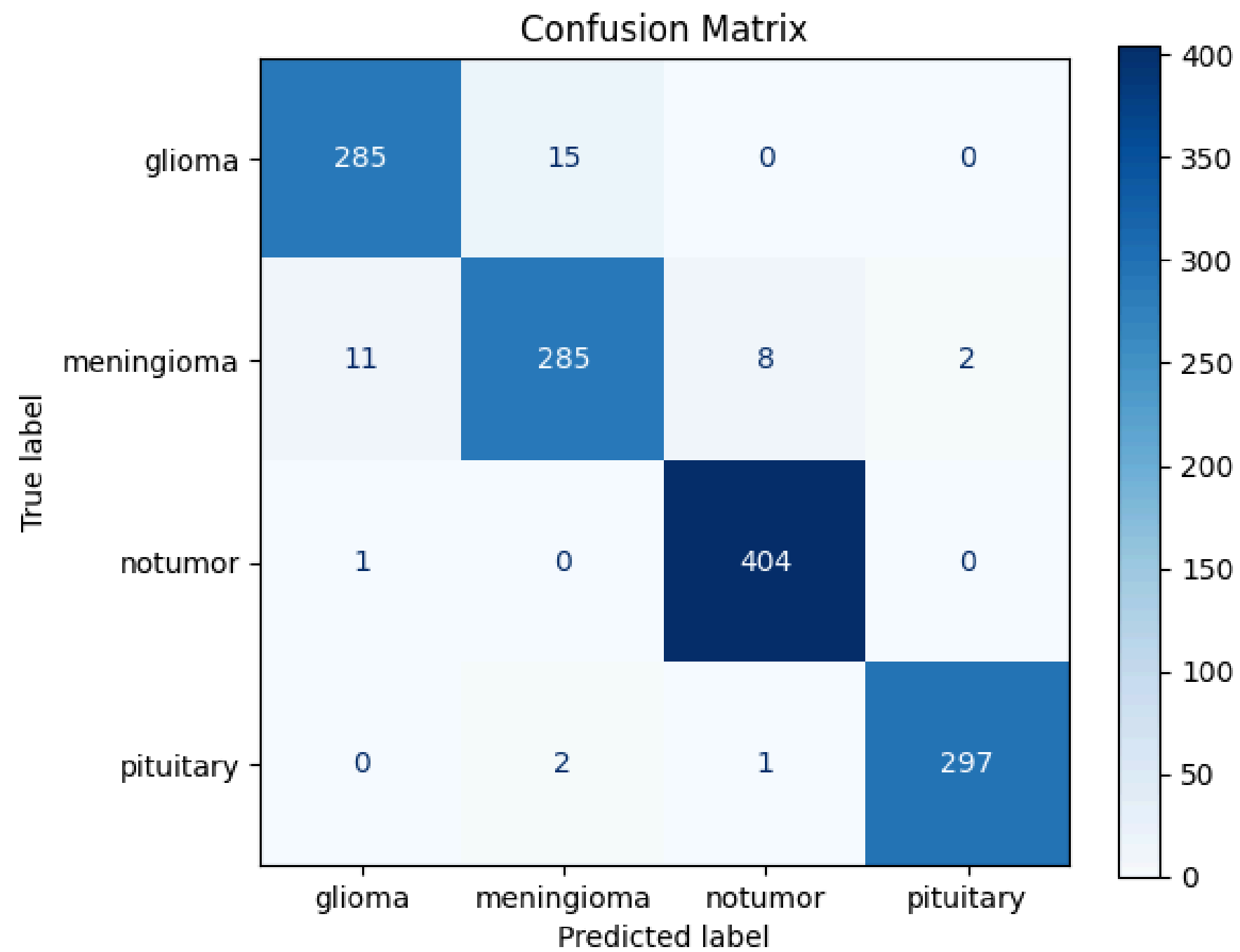
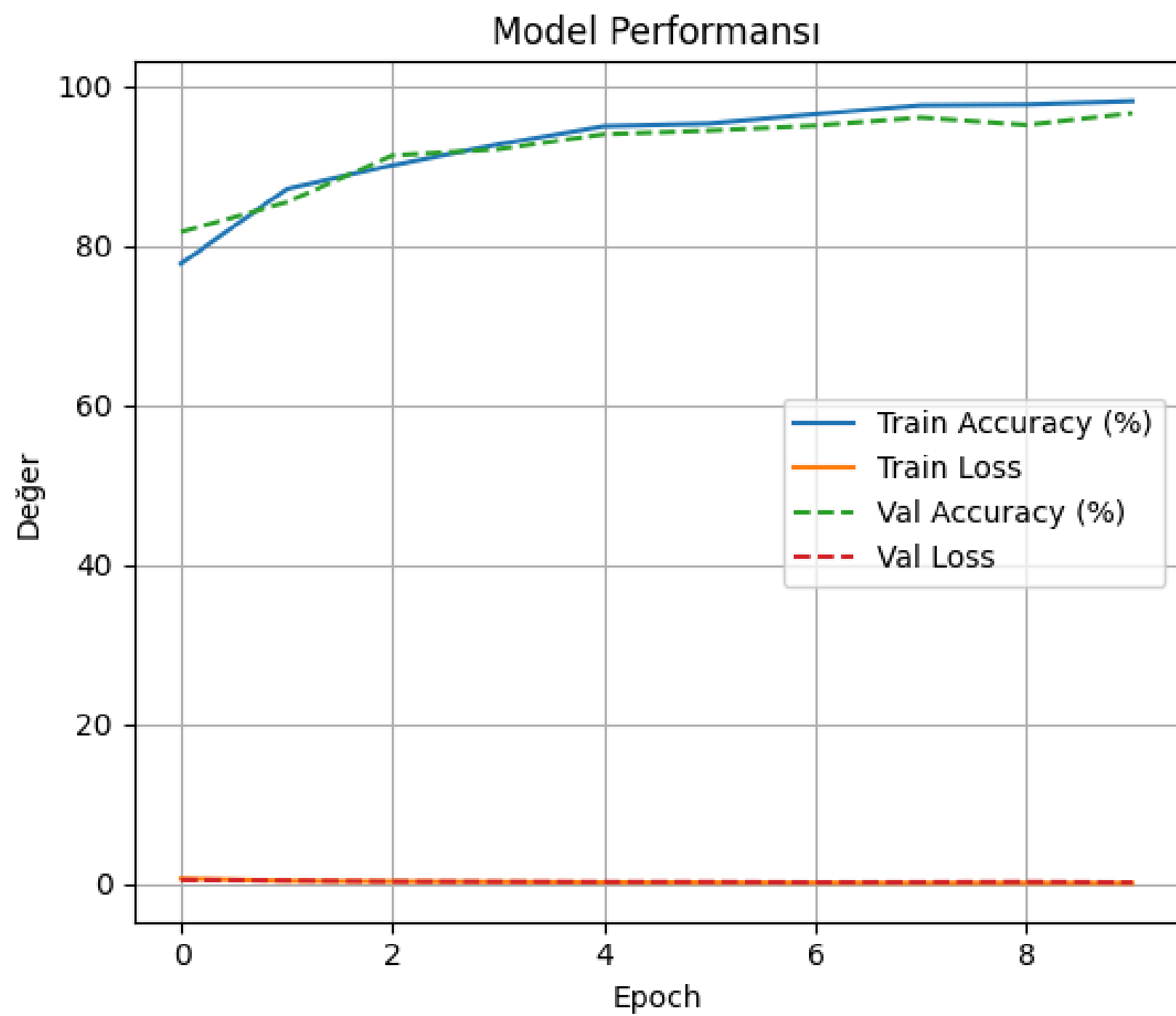
```
def train_model(model, train_loader, val_loader, num_epochs=10, lr=0.001, device="cpu"):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_acc_list = []
    train_loss_list = []
    val_acc_list = []
    val_loss_list = []
```

Değerlendirme

`evaluate.py`

- Test seti üzerinde tahmin yapar
- Accuracy hesaplar
- Confusion matrix oluşturur
Neden softmax + argmax?
- Model çıktıları doğrudan logits olduğu için, softmax ile sınıf olasılıkları hesaplanıp en yüksek skor alınır.



Model Kaydetme ve Kullanma

```
torch.save(model.state_dict(), f: "model_weights.pth")
```

- Eğitim tamamlandıktan sonra modelin ağırlıkları .pth dosyasına kaydedilir.
- Streamlit ve diğer inference modülleri bu ağırlığı kullanır.

Örnek Tahminler

inference.py

```
predict_and_show(model, test_loader, device=device, num_images=5)
```

Test setinden rastgele 10 MRI görüntüsü seçilir, tahmin yapılır ve gerçek sınıf ile karşılaştırılır. Görseller Matplotlib ile gösterilir.

C:\Users\USER\PycharmProjects\brain_tumor_detector\.venv\Scripts\python.exe\Scripts\python.exe C:\Users\USER\PycharmProjects\brain_tumor_detector

Epoch 1/10: 100%|██████████| 357/357 [03:04<00:00, 1.93it/s]

Epoch [1/10], Train Loss: 0.5985, Train Acc: 77.73%, Val Loss: 0.4422, Val Acc: 81.69%

Epoch 2/10: 100%|██████████| 357/357 [02:55<00:00, 2.03it/s]

Epoch 3/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [2/10], Train Loss: 0.3388, Train Acc: 87.10%, Val Loss: 0.3743, Val Acc: 85.43%

Epoch 3/10: 100%|██████████| 357/357 [02:53<00:00, 2.05it/s]

Epoch 4/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [3/10], Train Loss: 0.2574, Train Acc: 90.07%, Val Loss: 0.2447, Val Acc: 91.30%

Epoch 4/10: 100%|██████████| 357/357 [02:40<00:00, 2.22it/s]

Epoch [4/10], Train Loss: 0.2013, Train Acc: 92.68%, Val Loss: 0.2184, Val Acc: 92.07%

Epoch 5/10: 100%|██████████| 357/357 [02:45<00:00, 2.16it/s]

Epoch 6/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [5/10], Train Loss: 0.1424, Train Acc: 94.96%, Val Loss: 0.1878, Val Acc: 93.97%

Epoch 6/10: 100%|██████████| 357/357 [02:41<00:00, 2.21it/s]

Epoch 7/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [6/10], Train Loss: 0.1267, Train Acc: 95.29%, Val Loss: 0.1899, Val Acc: 94.43%

Epoch 7/10: 100%|██████████| 357/357 [02:37<00:00, 2.27it/s]

Epoch [7/10], Train Loss: 0.1027, Train Acc: 96.46%, Val Loss: 0.1406, Val Acc: 95.04%

Epoch 8/10: 100%|██████████| 357/357 [02:36<00:00, 2.28it/s]

Epoch 9/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [8/10], Train Loss: 0.0730, Train Acc: 97.60%, Val Loss: 0.1586, Val Acc: 96.03%

Epoch 9/10: 100%|██████████| 357/357 [02:38<00:00, 2.25it/s]

Epoch 10/10: 0%| | 0/357 [00:00<?, ?it/s]Epoch [9/10], Train Loss: 0.0692, Train Acc: 97.71%, Val Loss: 0.2000, Val Acc: 95.12%

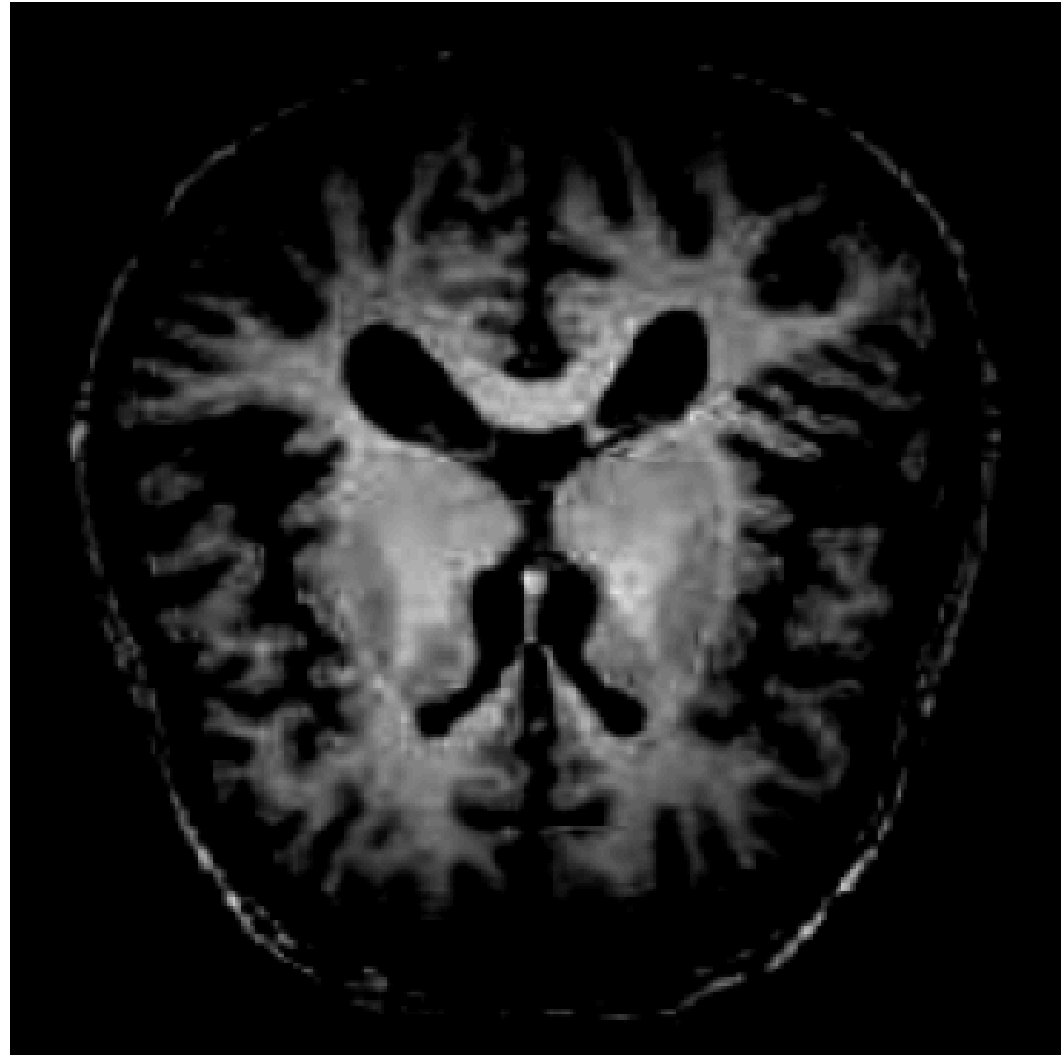
Epoch 10/10: 100%|██████████| 357/357 [02:43<00:00, 2.18it/s]

Epoch [10/10], Train Loss: 0.0590, Train Acc: 98.13%, Val Loss: 0.1281, Val Acc: 96.57%

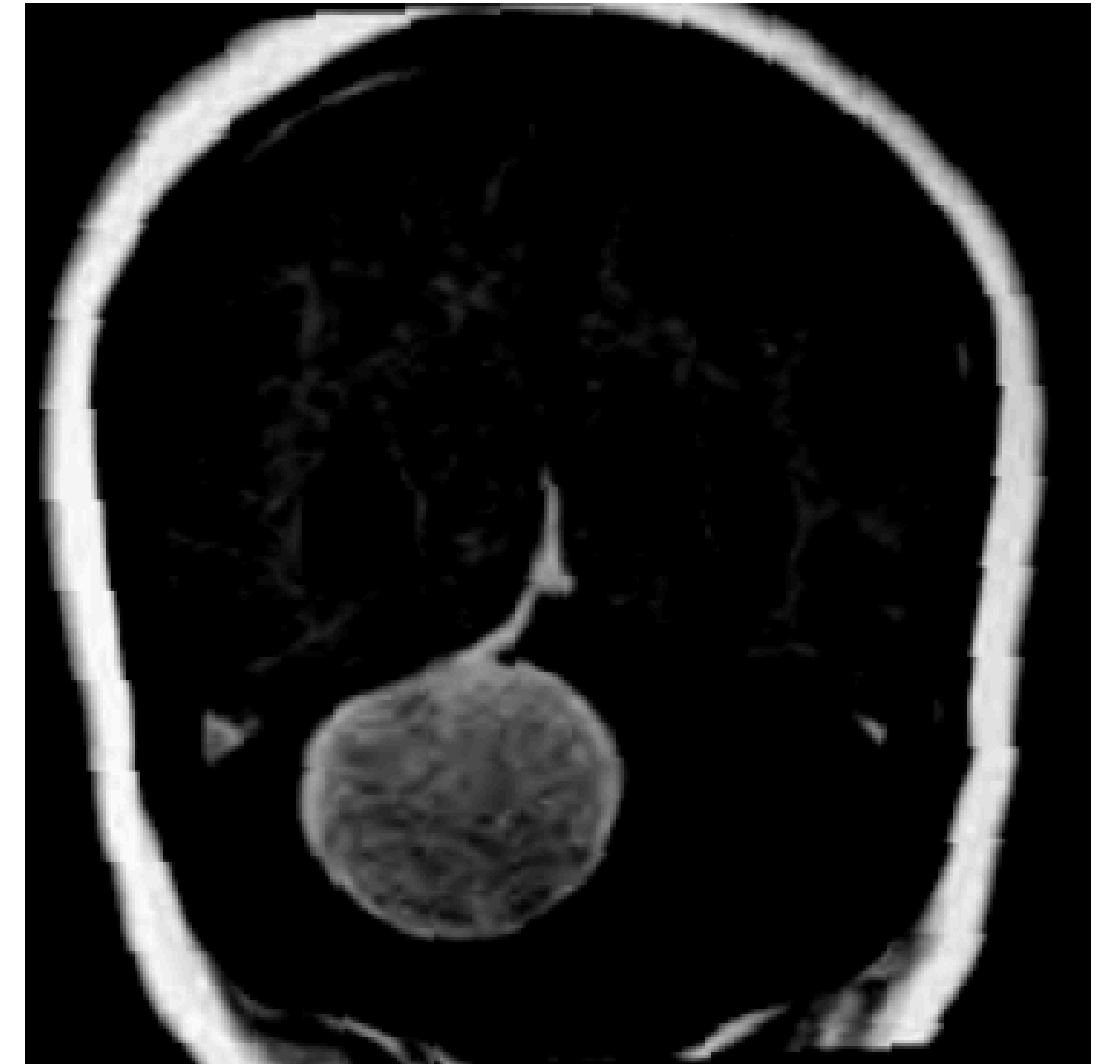
Eğitim tamamlandı.

✅ Model ağırlıkları 'model_weights.pth' dosyasına kaydedildi.

Tahmin: notumor | Gerçek: notumor



Tahmin: meningioma | Gerçek: meningioma



Grad-CAM ile Görselleştirme

app.py

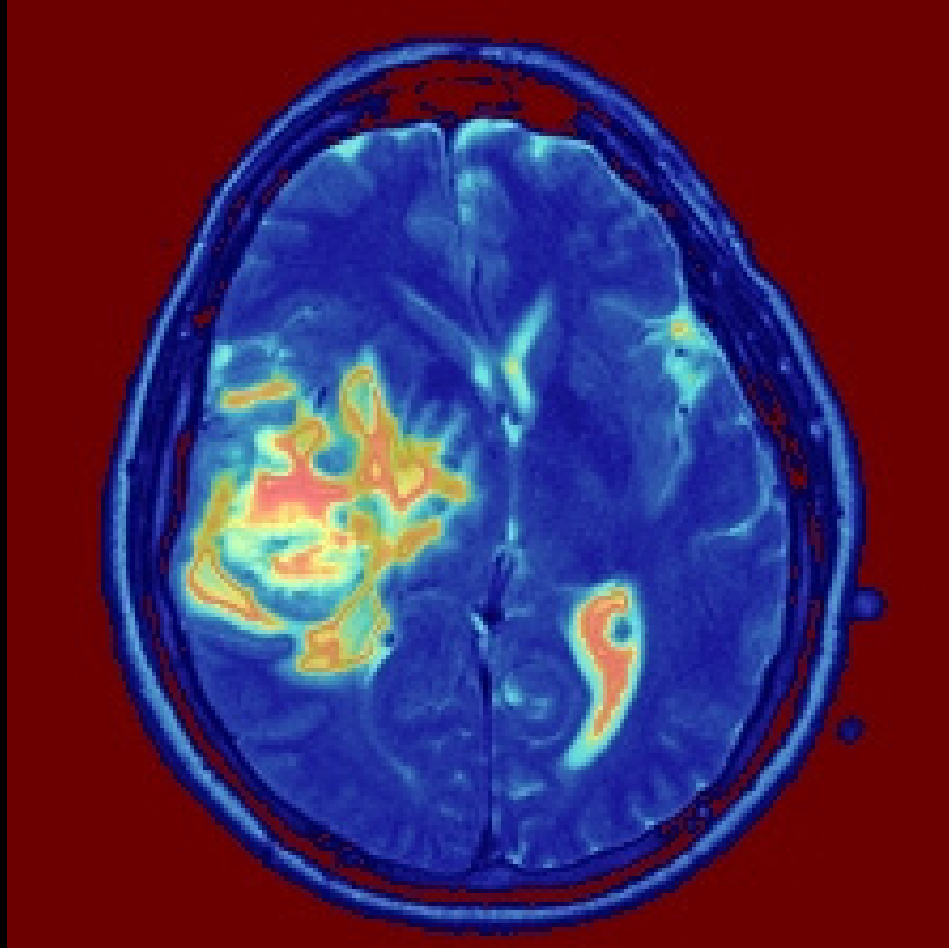
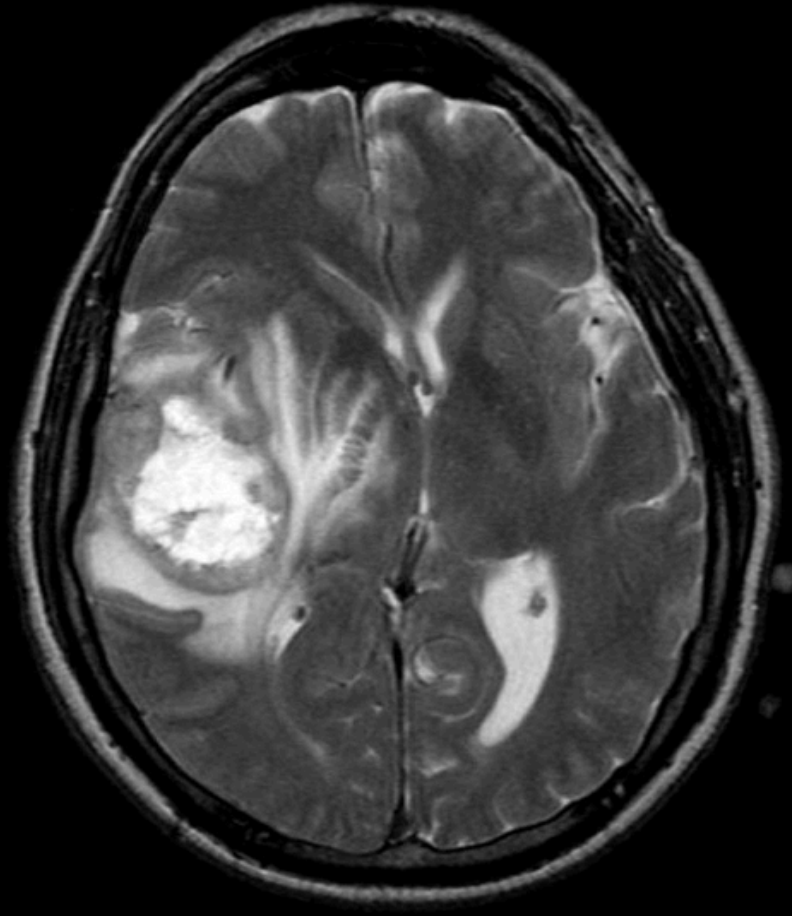
```
generate_gradcam(...)
```

Amaç:

Modelin karar verirken hangi bölgelere odaklandığını göstermek.

Nasıl Çalışır:

- conv2 katmanından aktivasyonlar ve gradyanlar alınır
- Kanal bazlı ağırlıklı ortalama ile önem haritası (CAM) oluşturulur
- MRI görüntüsü üzerine bu harita bindirilir





Arayüz: Streamlit ile Uygulama

app.py

st.file_uploader(...) → model(...) → Grad-CAM

Özellikler:

- Kullanıcı arayüzü üzerinden MRI yüklenebilir
- Model tahmin sonucu sınıf ismi ve olasılığı olarak gösterilir
- Grad-CAM görselleştirmesi ile birlikte

Avantajı:

Modelin çalışması kolayca test edilebilir ve yorumlanabilir hale gelir.



Beyin MRI Tümör Tespiti

Bir MRI görüntüsü yükleyin. Model tümör tipini tahmin edecek ve Grad-CAM ile görselleştirecek.



MRI Görüntüsü Seçin



Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files